CLASS AND OBJECTS

CLASS

- Class is a set of objects that shares common structure (variables) and behaviors (functions)
- It is a user defined type
- It contains both variables and functions
- Class is created using the reserved word named **class**
- By default, all the members in the class are **private modifier**.

OBJECT

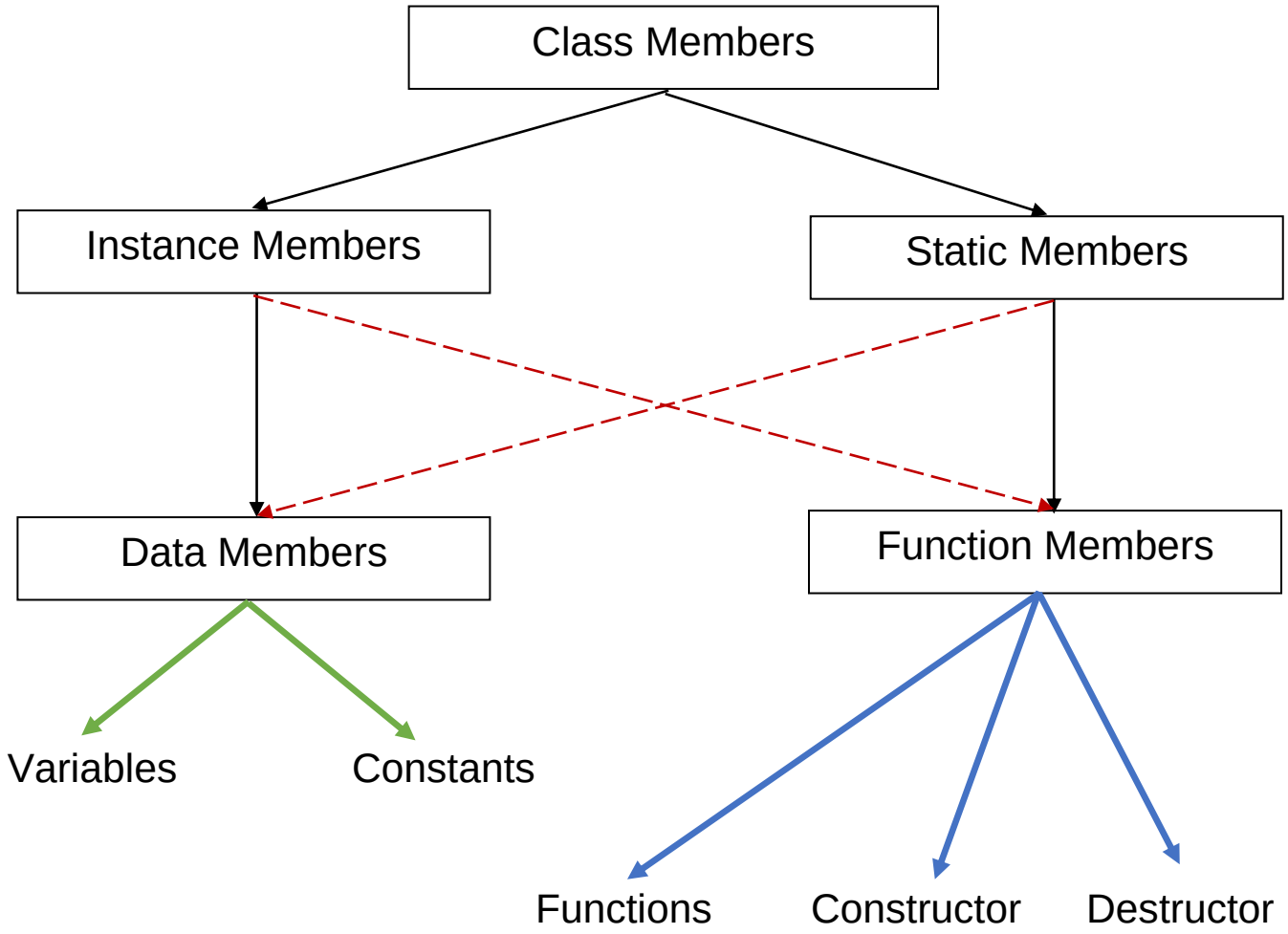
- It is an instance of a class
- It is used to access instance variables and methods
- It will be created automatically, when a constructor of the same class is created
- Memory will be created, when an object of a class is created
- Many objects are allowed per class.

CLASS DEFINITION

- The class definition contains two sections. They are:
    1. Variable section
    2. Function section

Syntax

```
class <user-defined name>
{
        // variable sections
        // function sections
};
```



CHARACTERISTICS OF A CLASS

- It must be defined using class keyword.
- It can contain variables and functions
- It can contain static variables & static methods
- It can be **derived** from another class (using colon **:** keyword)
- A class can be preceded with an access modifier.
- Private is a default modifier to class.

ADDING VARIABLES

- Any number of variables can be added to the class.
- These variables that are declared inside the class are called Instance / Static variables.
- The declaration of instance variables **doesn't occupy** any space in memory. At the time of creation, they will **occupy the memory** in some extent.

Syntax

```
<return-type> <name>;
<return-type> <name>=initial values;
```

Supported modifiers : (private, public, protected)

Example

// Heterogeneous variable declaration

```
char *name="Ganesh";
int id=99;
```

// Homogeneous variable declaration

```
int a=35, b=95, c=90;
```

ADDING FUNCTIONS

- Any number of functions can be added in a class.
- These functions that are defined inside a class is called **instance method.**

Usage

- It is used to provide the implementation of variables(data).

Syntax

```
<modifier>:
<r.type> <u.name>
{
        // user code
}
```

- Supported modifiers: (private, public, protected)

Example

```
public:
        void disp()
        {
                // user code
        }
```

OBJECT CREATION

- An object is instance of a class.
- Unlike java / c#, the c++ objects are **value type**. So, no need to use new modifier in the creation of objects.
- An object is a block of memory that contains the space (memory) to store all the instance variables
- Memory will be allocated when objects are created in the class
- It can be local object or global object.

Two Ways of creating an object

- In c++, we can create an object using two ways. They are:
    1. Simple declaration
    2. Pointer

1. SIMPLE DECLARATION (NORMAL OBJECT)

- This is actually a variable (object) not pointer
- **Static object**
- It will be stored on stack memory. Because **this is value type**
- It will be automatically destroyed when it goes out of the scope
- Default constructor will be called when object is created.

Syntax

```
<class-name> <object-name>;
```

2. POINTER (POINTER OBJECT)

- This is dynamic method of creating an object with help of new operator
- Dynamic object
- Here object **will not be destroyed when it goes out of the scope**. The “delete <object-name>” operator is explicitly used to delete the objects
- This type of an object will be **stored on heap memory**
- Just returns a pointer.

Syntax

```
<class-name>  *<object-name>=new <constructor-name>();

                    (OR)

<class-name>  *<object-name>=new <constructor-name>;
```

Example

```
class Test {…};
.
.
.
Test *obj=new Test();

Test *obj=new Test;
```

// Dynamic object with parameterized constructor

// Dynamic object with default constructor

DIFFERENCE BETWEEN STATIC AND DYNAMIC OBJECT

S.N	NORMAL OBJECT	POINTER OBJECT
1.	Static object	Dynamic object
2.	This object is created without using new modifier	This object is created with help of new modifier
3.	This object is value type. So, it will be stored on stack memory	This object is reference type based. So, it will be stored on heap memory
4.	No need to use constructor name in the object creation **Example** <class-name> <object-name>;	Constructor name is needed in the object creation **Example** <class-name> <object-name>= new <constructor-name>;
5.	Here object will be automatically destroyed when it goes out of the scope	Here object will not be automatically destroyed. The **delete operator** must be used to delete the object

NOTE

- It is an important to note that, when creating an object with **new modifier**, we always have to remember deleting it.

ACCESSING CLASS MEMBERS

- The **instance** data members (variables) and member functions (functions) of a class is accessed using object with **dot ('.') operator**
- The **static** data members (variables) and member functions (functions) of a class is accessed using class-name with **dot ('.') operator**

Syntax

```
<object-name>.variable-name;

<object-name>.function-name();
```

Example

```
obj.name="ragul";

obj.player("laxman");
```

// Accessing instance variable

// Accessing instance function.

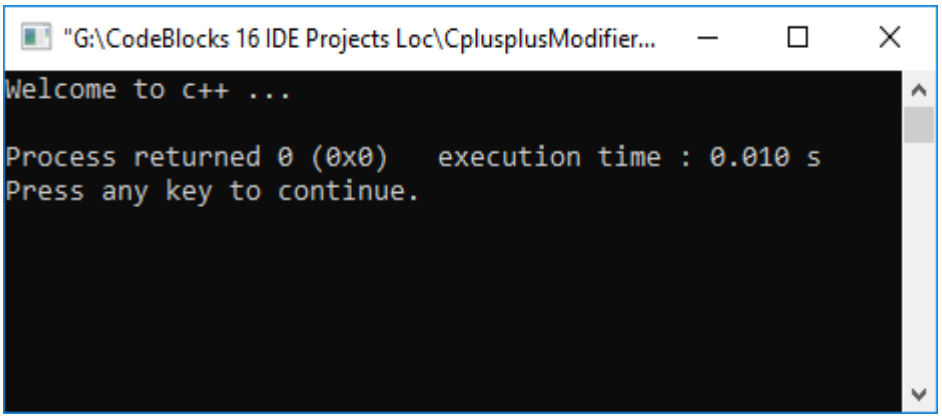
I. EXAMPLE OF CLASS (LOCAL OBJECT)

1. SOURCE CODE

```
#include<iostream>
using namespace std;                    // global namespace std.
class Welcome
{
        public:                         // public method
        void message()
        {
            cout<<"Welcome to c++ ...\n";
        }
};
int main()
{
   Welcome obj;      // local object creation (with default constructor)
   obj.message();    // calling instance method using object
   return 0;
}
```

Public Method

2. OUTPUT



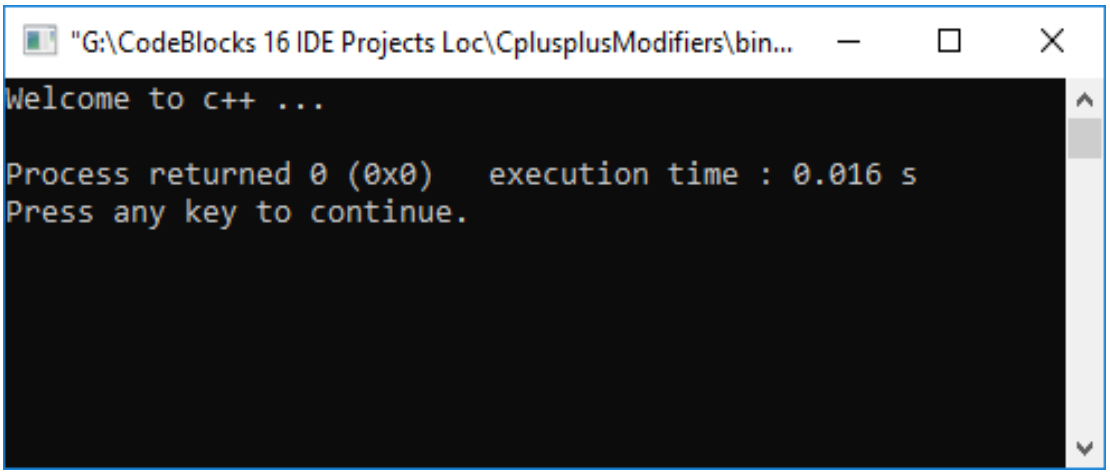
II. EXAMPLE OF CLASS (GLOBAL OBJECT)

1. SOURCE CODE

```
#include<iostream>
using namespace std;                    // global namespace
class Welcome
{
        public:
        void message()
        {
            cout<<"Welcome to c++ ...\n";
        }
} obj;
int main()
{
    obj.message();     // calling instance method using object
    return 0;
}
```

global object creation (with default constructor)

2. OUTPUT



DEFINITION OF MEMBER FUNCTION

- C++ provides two options for defining a function in a class. They are:
    1. Class with inside the member function
    2. Class with outside the member function

1. Class with inside the member function

- Here function definitions are placed in a class.

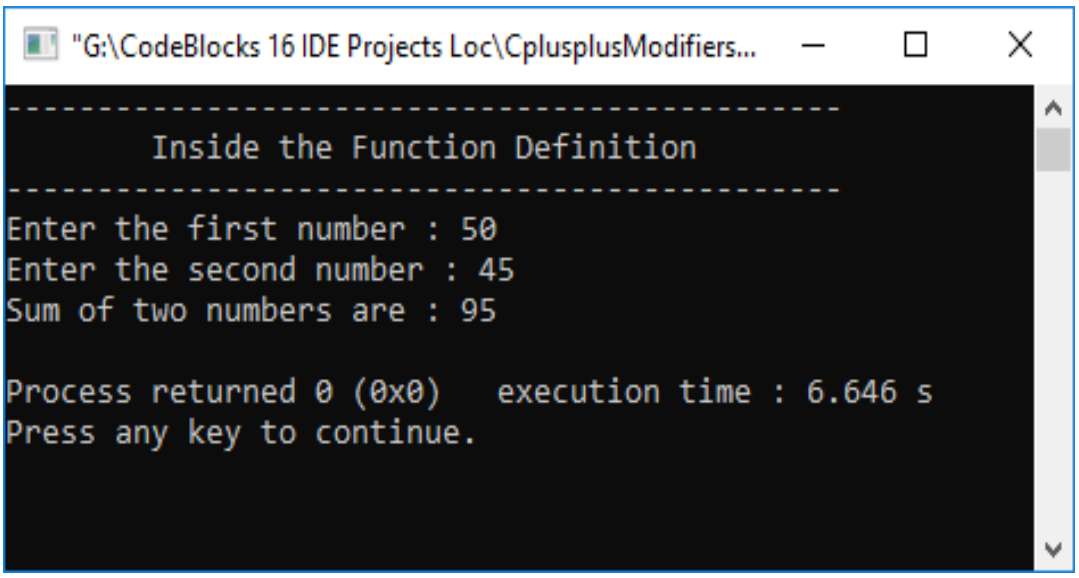
III. CLASS WITH INSIDE MEMBER FUNCTION

1. SOURCE CODE

```
#include<iostream>
using namespace std;
class Test
{
public:
   int a,b,c;
   void input()
   {
      cout<<"Enter the first number : ";
      cin>>a;
      cout<<"Enter the second number : ";
      cin>>b;
      c=a+b;
   }
   void output()
   {
```

```
        cout<<"Sum of two numbers are : "<<c<<"\n";
    }
};
int main()
{
    cout<<"-------------------------------------------\n";
    cout<<"\tInside the Function Definition\n";
    cout<<"-------------------------------------------\n";
    Test obj;
    obj.input();
    obj.output();
    return 0;
}
```

2. OUTPUT



2. Class with outside the member function

- It is an optional
- Here function definitions are placed outside of a class
- This is done by using the **scope resolution operator (::)**

Syntax

```
<r.type> <class-name>::<function-name>(args)
{
        // user code
}
```

Example

```
class Test
{
        public:
                void disp();
};

void Test::disp()
{
        // user code
}
```

Function Declaration

Function Definition

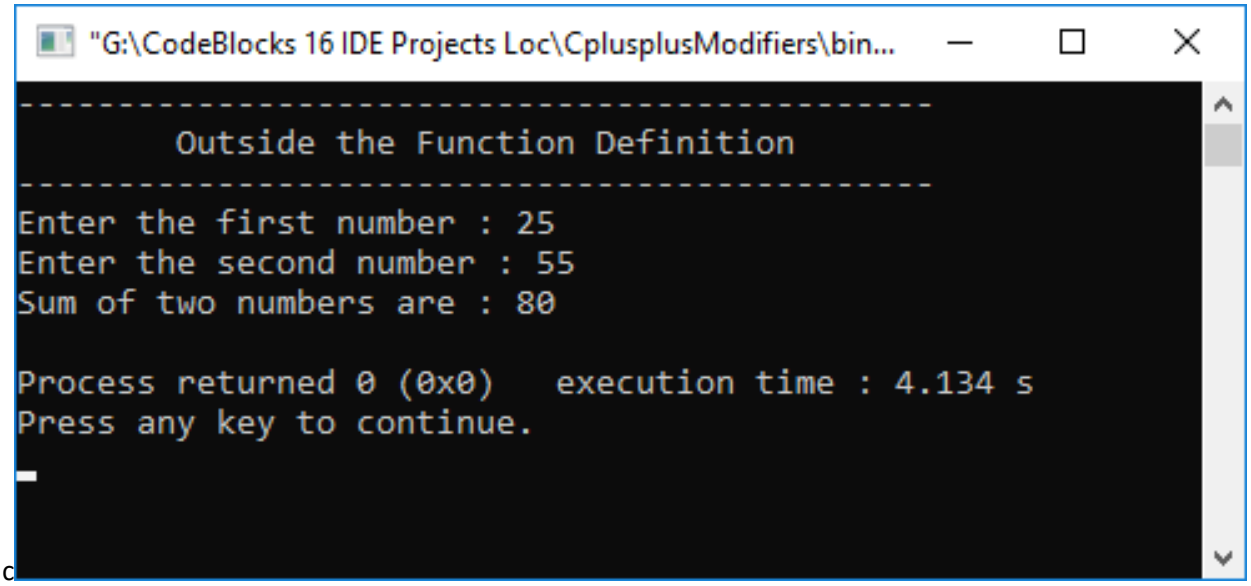
IV. CLASS WITH OUTSIDE MEMBER FUNCTION

1. SOURCE CODE

```
#include<iostream>
using namespace std;
class Test
{
        public:
        int a,b,c;
        void input();
        void output();
};
void Test::input()
{
      cout<<"Enter the first number : ";
      cin>>a;
      cout<<"Enter the second number : ";
      cin>>b;
      c=a+b;
}
void Test::output()
{
      cout<<"Sum of two numbers are : "<<c<<"\n";
}
```

```
int main()
{
    cout<<"-------------------------------------------\n";
    cout<<"\tOutside the Function Definition\n";
    cout<<"-------------------------------------------\n";
    Test obj;                    // local object
    obj.input();                 // calling instance method via object
    obj.output();                // calling instance method via object
    return 0;
}
```

2. OUTPUT



STATIC MEMBERS

- The static members are static variables, static functions, static objects, etc, …
- The static members are created using **static keyword**
- A static data member is shared by all objects of the class
- The static members are **stored on the static storage area**, not stack memory
- Static keyword can be used with following:
    1. Static variable
    2. Static member variable
    3. Static member function
    4. Static class objects.

1. Static variable

- If a variable is defined with the static keyword, that is called as static variable
- Unlike local variable, it is **initialized only once**.
- It is stored on the static storage area, not stack memory
- Default value is zero
- It has two types. They are: local static variable and global static variable

Local Static Variable

- If a variable is defined **inside of a function with static modifier**, that is called as local static variable
- Lifetime: till end of same function

Global Static Variable

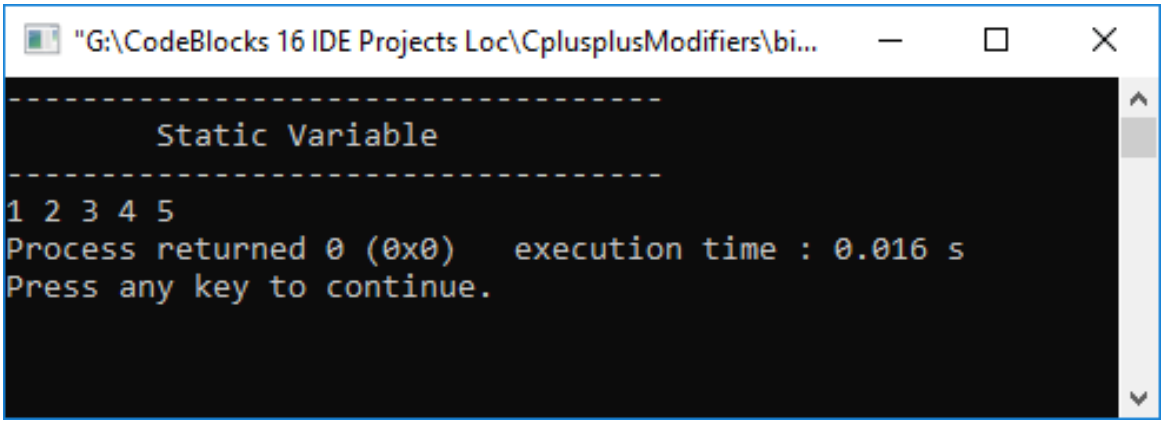
- If a variable is defined **outside of a function with static modifier**, that is called as global static variable
- Lifetime: **till end of the program.**

V. STATIC VARIABLE
(LOCAL STATIC VARIABLE)

1. SOURCE CODE

```
#include<iostream>
using namespace std;
void disp()
{
        static int k=1;          // static variable definition
        cout<<k<<" ";
        k++;                     // static variable is initialized only once
}
int main()
{
        cout<<"----------------------------------\n";
        cout<<"\tStatic Variable\n";
        cout<<"----------------------------------\n";
        for(int i=0;i<5;i++)
        {
            disp();
        }
        return 0;
}
```

2. OUTPUT



VI. LOCAL VARIABLE

1. SOURCE CODE

```
#include<iostream>
using namespace std;
void disp()
{
        int k=1;                    // local variable definition
        cout<<k<<" ";
        k++;                        // local variable is initialized every time
}
int main()
{
        cout<<"----------------------------------\n";
        cout<<"\tStatic Variable\n";
        cout<<"----------------------------------\n";
        for(int i=0;i<5;i++)
        {
            disp();
        }
```

```
        return 0;
}
```

2. OUTPUT

```
"G:\CodeBlocks 16 IDE Projects Loc\CplusplusModifiers\bin...
-----------------------------------
        Local Variable
-----------------------------------
1 1 1 1 1
Process returned 0 (0x0)   execution time : 0.016 s
Press any key to continue.
```

DIFFERENCE BETWEEN LOCAL AND STATIC VARIABLE

S.N	LOCAL VARIABLE	STATIC VARIABLE
1.	It is created without using static modifier	It is created with help of static modifier
2.	It is initialized **many times**	It is initialized **only once**
3.	It is stored on the stack memory	It is stored on the static storage area
4.	It must be defined inside a function	It can be defined **inside or outside** of a function
5.	Default value: Garbage value	Default value: **0**

2. Static Member variable

- If a variable is defined inside the class with the static keyword, that is called as static member variable
- Only one copy of the static variable is shared by all the objects of a class. Because static member variable purely depends on class
- Unlike instance variable, it does support direct initialization.

Example 1

```
class Test
{
        public:
                static int k=20;        // Error
};
```

- Static member variables are **not initialized using constructor, static functions.**

Example 2

```
class Test
{
        public:
                static int k;
                Test()
                {
                k=25;                   // Error
                }
};
```

Example 3

```
class Test
{
        public:
                static int k;
                static void disp()
                {
                        k=99;           // Error
                }
};
```

- The initialization of **static variable is not permitted in constructor**, static functions and instance functions
- The static variables are only initialized outside of a class using scope resolution operator.

Syntax

```
<return type> <class-name>::<variable-name> = initial value;
```

Example

```
int Test::a=25;
```

Return type Class name Variable Name Value

Accessing (Calling)

- To access the static variable, there is no need of creating the object of the class.
- The static member variable and static functions are accessed by using class name with scope resolution operator (::)
- **Scope**: till end of the class.

Syntax

```
<class-name>::<static variable-name>;
<class-name>::<static function-name>();
```

Example

```
Test::x=25;          // Accessing static variable
Test::disp();        // Accessing static function
```

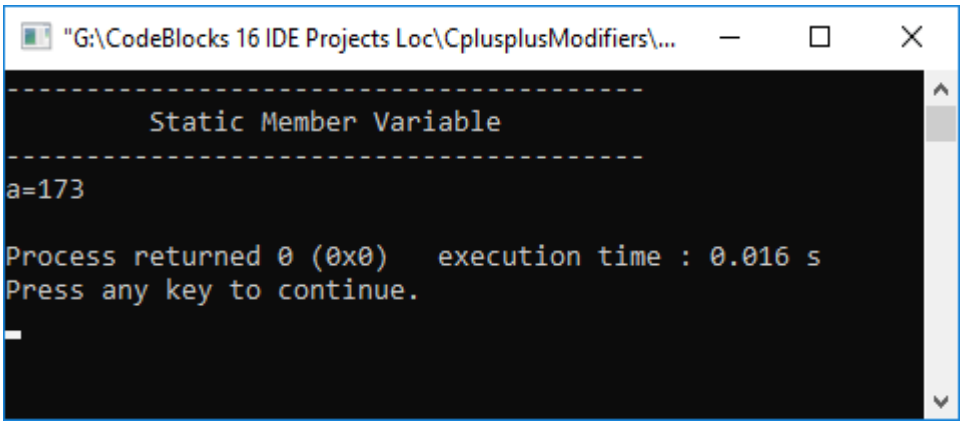
VII. STATIC MEMBER VARIABLE

1. SOURCE CODE

```
#include<iostream>
using namespace std;      // global namespace
class Test
{
public:
    static int a;         // static variable declaration
    static void disp()    // static function
    {
        cout<<"a="<<a<<endl;
    }
};
int Test::a=173;          // static variable initialization using scope
                          resolution operator
int main()
{
    cout<<"--------------------------------------\n";
    cout<<"\t Static Member Variable\n";
    cout<<"--------------------------------------\n";
    Test::disp();         // static method is called using class name with
                          scope resolution operator
    return 0;
```

}

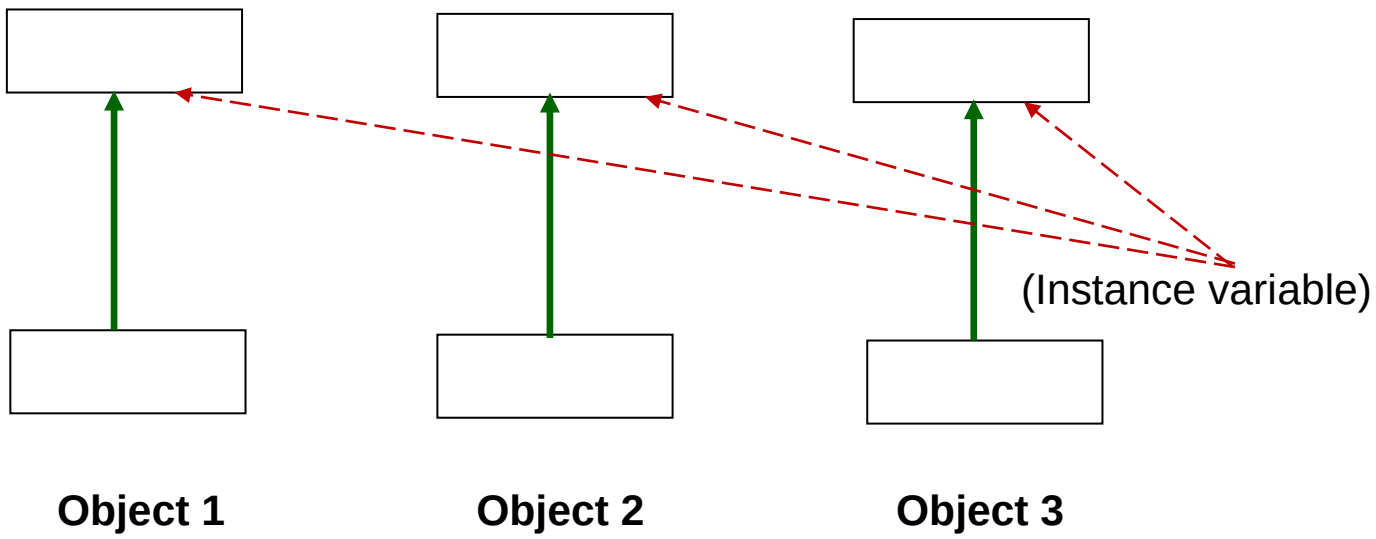
2. OUTPUT



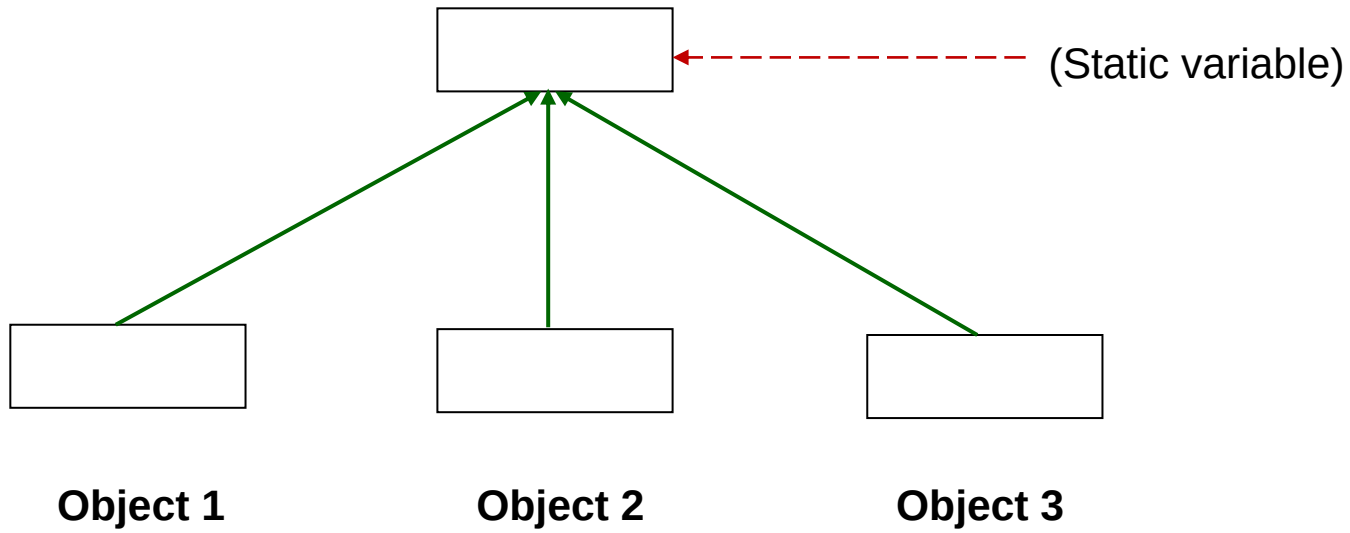
COMBINATION OF INSTANCE & STATIC VARIABLES

- If a variable is declared with standard modifiers such as private | public | protected (without static modifier) that is called as instance variable
- If a variable is declared with static modifier that is called as static variable or static member variable
- Separate copy of an instance variable is allocated to all the objects of a class. But only one copy (common copy) of the static variable is shared between all the objects.
- Any changes in the instance variable of a particular object will not affect the values of instance variables of other objects. But any changes in the static variable must reflect the values of static variables of all the objects (entire class)

INSTANCE VARIABLE



STATIC VARIABLE



VIII. COMBINING INSTANCE AND STATIC VARIABLE

1. SOURCE CODE

```
#include<iostream>
using namespace std;
class Test
{
        public:
            int k;                  // instance variable declaration
            static int m;           // static variable declaration
        Test()
        {
            init();                 // instance variable definition
            k=k+50;                 // instance variable modification
            m=m+100;                // static variable modification
         }
        void init()
        {
            k=30;
        }
         void disp()
        {
            cout<<"k= "<<k<<"\n";
            cout<<"m= "<<m<<"\n";
        }
};
int Test::m=10;                     // static variable definition
```

```
int main()
{
    cout<<"-------------------------------------------------------\n";
    cout<<"\t Instance vs Static Variables\n";
    cout<<"-------------------------------------------------------\n";
    Test t1,t2,t3;
    cout<<"\t\tObject 1\n";
    t1.disp();
    cout<<"-------------------------------------------------------\n";
    cout<<"\t\tObject 2\n";
    t2.disp();
    cout<<"-------------------------------------------------------\n";
    cout<<"\t\tObject 3\n";
    t3.disp();
    cout<<"-------------------------------------------------------\n";
    cout<<"Accessing static variable using objects: "<<t1.m<<" "<<t2.m<<"
"<<t3.m<<"\n";
    cout<<"Accessing static variable using class: "<<Test::m<<"\n";
    cout<<"-------------------------------------------------------\n";
    return 0;
}
```

2. OUTPUT

```
"G:\CodeBlocks 16 IDE Projects Loc\CplusplusModifiers\bin\Deb...
-------------------------------------------------------
        Instance vs Static Variables
-------------------------------------------------------
                Object 1
k= 80
m= 310
-------------------------------------------------------
                Object 2
k= 80
m= 310
-------------------------------------------------------
                Object 3
k= 80
m= 310
-------------------------------------------------------
Accessing static variable using objects: 310 310 310
Accessing static variable using class: 310
-------------------------------------------------------

Process returned 0 (0x0)   execution time : 0.016 s
Press any key to continue.
```

DIFFERENCE BETWEEN LOCAL AND STATIC VARIABLE

S.N	INSTANCE VARIABLE	STATIC VARIABLE
1.	It is created without using static modifier	It is created with help of static modifier

2.	It supports direct initialization
	Ex.
	class Test
	{
	public:
	int a=25; // legal
	};
		It does not support direct initialization
		Ex.
		class Test
		{
		public:
		static int a=25; **// illegal**
		};

3.	It is stored on the stack memory	It is stored on the static storage area
4.	It can be initialized inside of a class	It can be initialized outside of a class
5.	It is accessed with help of object (It depends on object)	It is access with help of class (It depends on class)

3. STATIC FUNCTION (STATIC MEMBER FUNCTIONS)

- If a **function is defined with the static keyword**, that is called as static function
- It is used **to access only static member variable and other static functions**
- It purely depends on class
- It is called by using class-name with scope resolution operator
- Like static data member, the static member function is also a class function. It is not associated with any class object
- Static function work for the class as whole rather than for a particular object of a class
- It is not possible to access Instance variable (Non static variable) through static function

Example

```
class Test
{
public:
    int b;
    static void disp()
    {
        b=20;            // Error
    }
```

- Instance variable can' be accessed inside of a static function

Accessing Static function

- Static members are purely depending on class
- Static function can be called using class name with scope resolution operator
- Also. it is possible to access static function using object with dot (.) operator

Syntax

```
<class-name>::<static function-name>();
```

Example

```
Test::disp();
```

// Accessing static function

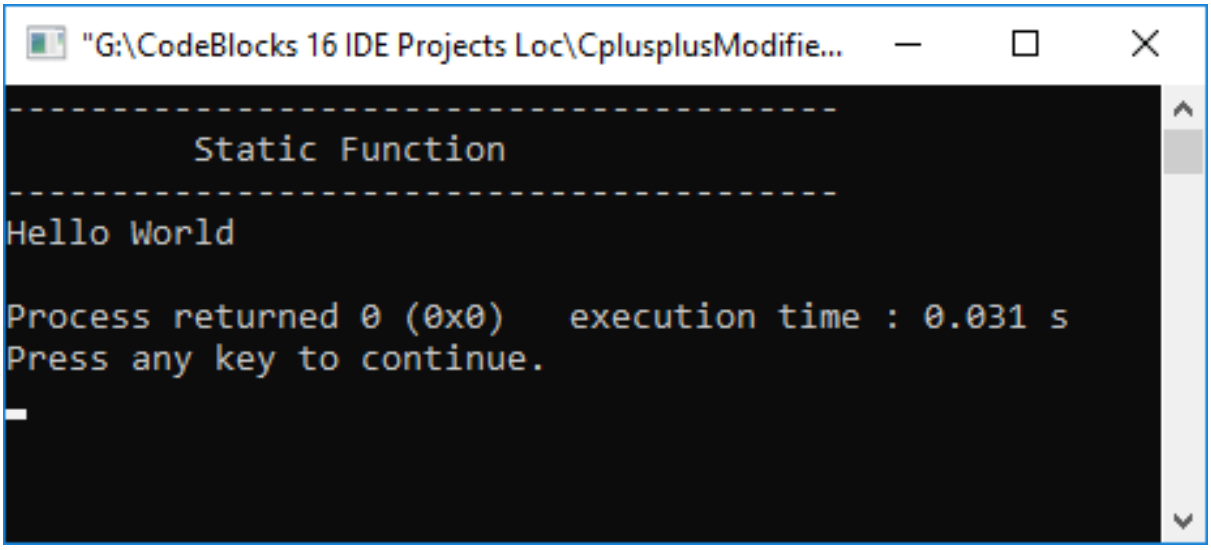
IX. STATIC MEMBER FUNCION

1. SOURCE CODE

```
#include<iostream>

using namespace std;

class Test

{

public:

        static void disp()              // static function

        {

            cout<<"Hello World\n";

        }

};
```

```
int main()
{
    cout<<"--------------------------------------\n";
    cout<<"\t Static Function\n";
    cout<<"--------------------------------------\n";
    Test::disp();                    // calling static function using class-name
    return 0;
}
```

2. OUTPUT



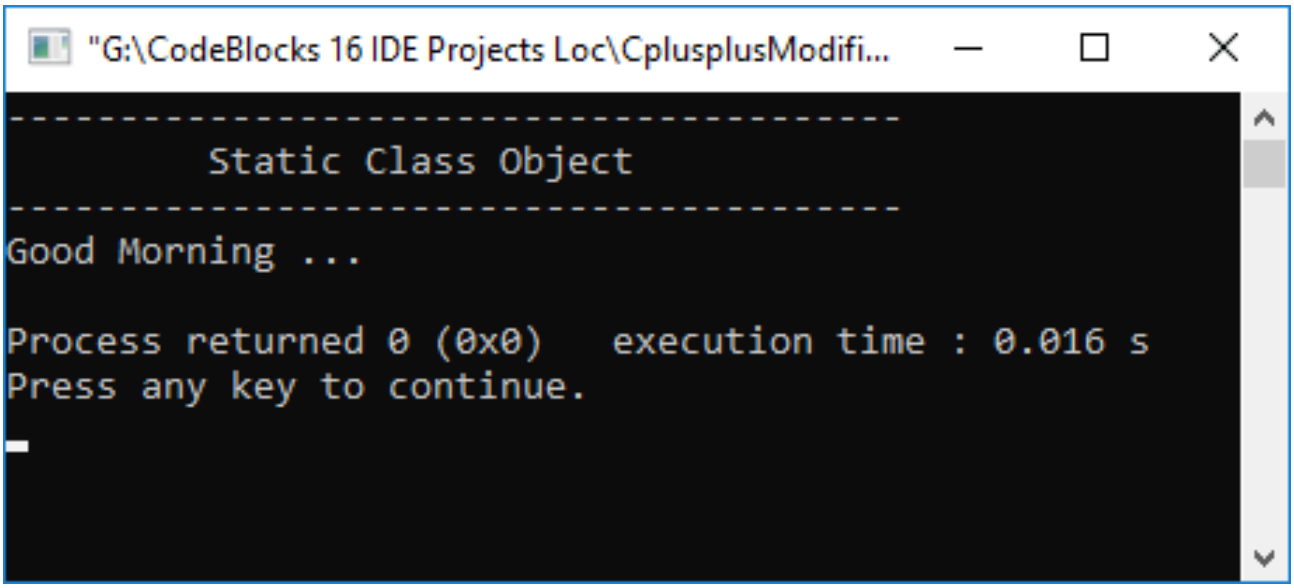
4. Static Object

- It is optional
- Unlike local object, it has scope till the program lifetime
- If object is defined with static modifier, that is called as static class object
- Static objects are initialized using constructor like other normal objects
- It works in the same way for class objects too.
- It is **allocated in the static storage area and have scope till end of program**
- It has two types: They are
    1. Local static object :
        - It has local access & scope till end of the program
    2. Global static object
        - It has global access & scope till end of the program

X. LOCAL STATIC OBJECT

1. SOURCE CODE

```
#include<iostream>

using namespace std;

class Test

{

public:

        void disp()

        {

            cout<<"Good Morning ...\n";

        }

};
```

```
int main()
{
        cout<<"--------------------------------------\n";
        cout<<"\t Static Class Object\n";
        cout<<"--------------------------------------\n";
        static Test obj;                    // local static object
        obj.disp();
        return 0;
}
```

2. OUTPUT